



UADY
FACULTAD DE
MATEMÁTICAS

Universidad Autónoma de Yucatán

Facultad de Matemáticas

Ingeniería de software asistida por IA

ADA 02

Reverse Engineer the Agent Loop

Alumna:

Ashley Shaden Aguilar Moreno

Profesor:

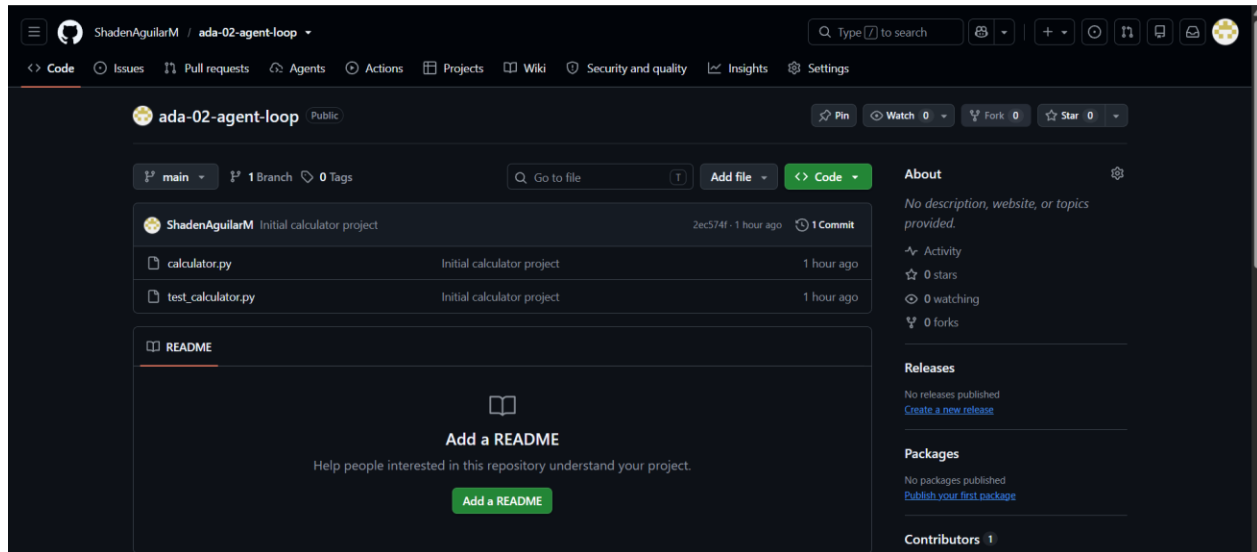
Sergio David Franco Garduño

Fecha de entrega:

03 de septiembre de 2026

1. Repositorio creado.

Link: <https://github.com/ShadenAguilarM/ada-02-agent-loop.git>



2. Línea base (Resultado de la prueba antes de usar el agente).

```
PS C:\Users\ashle\ada-02-agent-loop> pytest
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\ashle\ada-02-agent-loop
collected 4 items

test_calculator.py ...F [100%]

===== FAILURES =====
_____ test_divide _____

    def test_divide():
>     assert divide(10, 2) == 5
E       assert 20 == 5
E       + where 20 = divide(10, 2)

test_calculator.py:13: AssertionError
===== short test summary info =====
FAILED test_calculator.py::test_divide - assert 20 == 5
===== 1 failed, 3 passed in 0.10s =====
PS C:\Users\ashle\ada-02-agent-loop>
```

3. Experimento 1 — Observación sin modificar.

```
Antigravity CLI 1.1.22
aguilarmorenoashleyshaden01@gmail.com (Antigravity Starter Quota)
Gemini 3.7 Flash (High)
~/ada-02-agent-loop

> Inspect this repository.
Do not modify any files.
Tell me:
1. What files exist?
2. What does each file do?
3. What commands would you run to validate the project?
4. What potential problem do you see?

• Find(List all files in repo)
• ListDir(~/ada-02-agent-loop)
• Read(~/ada-02-agent-loop/calculator.py)
• Read(~/ada-02-agent-loop/test_calculator.py) (ctrl+o to expand)

### 1. Files in the Repository

• calculator.py
• test_calculator.py
• (Generated/cache folders: .git/, .pytest_cache/, __pycache__/)

### 2. What Each File Does

• calculator.py: Defines basic arithmetic functions:
  • calculator.py:1-2: Returns the sum of a and b.
  • calculator.py:4-5: Returns the difference of a and b.
  • calculator.py:7-8: Returns the product of a and b.
  • calculator.py:10-11: Intended to perform division of a by b.
• test_calculator.py: Contains unit tests for the arithmetic functions:
  • test_calculator.py:3-4: Tests that add(2, 3) == 5.
  • test_calculator.py:6-7: Tests that subtract(5, 3) == 2.
  • test_calculator.py:9-10: Tests that multiply(4, 3) == 12.
  • test_calculator.py:12-13: Tests that divide(10, 2) == 5.

### 3. Commands to Validate the Project

To execute the test suite:

    pytest

or

    python -m pytest

### 4. Potential Problems

1. **Bug in calculator.py:10-11**:
In calculator.py:10-11, the calculator.py:10-11 function executes return a * b instead of return a / b.
2. Test Failure:
Running pytest will fail on test_calculator.py:12-13 because divide(10, 2) currently evaluates to 20 instead of 5.
3. No Division-by-Zero Handling:
calculator.py:10-11 does not explicitly handle or validate cases where b == 0.
4. Missing Project Metadata:
There is no .gitignore file or dependency specification (e.g., pyproject.toml or requirements.txt) tracking dependencies like pytest.
```

4. Preguntas de observación.

- **¿Qué información tuvo que leer el agente?** Primero utilizó la función Find para rastrear los archivos dentro del repositorio y encontró 5 coincidencias. Después, utilizó el comando ListDir en la carpeta raíz ~/ada-02-agent-loop para conocer cómo estaba organizada. Descubrió que el repositorio contiene 2 archivos y 3 directorios.

Posteriormente, abrió el archivo `calculator.py` utilizando el comando `Read` y leyó sus 11 líneas. De igual forma, mediante el comando `Read`, leyó el archivo `test_calculator.py`, que contenía 13 líneas.

- **¿Qué herramienta utilizó para conocer el contenido del archivo?** Utilizó la herramienta `Read`, que le permitió acceder y leer el contenido de los archivos del repositorio.
- **¿Modificó algún archivo?** No. La instrucción indicaba explícitamente "Do not modify any files". El agente únicamente listó los archivos y leyó su contenido. No ejecutó ninguna operación de escritura ni realizó modificaciones en el código.
- **¿Cómo podría comprobar que su hipótesis sobre el bug es correcta?** Podría ejecutar las pruebas del proyecto utilizando `pytest` o `python -m pytest`. La hipótesis es que la función `divide()` está utilizando `*` en lugar de `/`. Por lo tanto, la prueba `assert divide(10, 2) == 5` debería fallar, ya que la implementación actual produciría `10 * 2 = 20`. Si `pytest` reporta que se esperaba 5, pero se obtuvo 20, entonces la hipótesis sobre el bug quedaría comprobada.
- **¿Qué diferencia observas entre pedir una respuesta a un LLM y permitir que un agente interactúe con un repositorio?** La diferencia principal es que un LLM puede razonar sobre la información que se le proporciona, mientras que un agente puede interactuar directamente con un entorno para obtener información y realizar acciones. Si a un LLM no se le proporciona el código, no puede inspeccionarlo directamente. En cambio, el agente no solamente generó una respuesta: primero interactuó con el repositorio, obtuvo información mediante herramientas, analizó su contenido y propuso una forma de verificar su conclusión.

5. Experimento 2 — Resolver el bug.

```
> Find and fix the bug in the calculator.
Requirements:
- Do not modify the tests.
- Inspect the relevant files first.
- Run the tests before making changes.
- Make the smallest change necessary.
- Run the tests again after the change.
- Explain what you changed.

• Read(~/.ada-02-agent-loop/calculator.py)
  L Read 11 lines
• Read(~/.ada-02-agent-loop/test_calculator.py)
  L Read 13 lines
• Bash(pytest)
  L <output +9 lines>

      def test_divide():
      >     assert divide(10, 2) == 5
      E     assert 20 == 5
      E     + where 20 = divide(10, 2)

      test_calculator.py:13: AssertionError
      ===== short test summary info =====
      FAILED test_calculator.py::test_divide - assert 20 == 5
      ===== 1 failed, 3 passed in 0.13s =====

• Edit(~/.ada-02-agent-loop/calculator.py)
  L +1 / -1 lines
      8         return a * b
      9
      10     def divide(a, b):
      11 -         return a * b
      11 +         return a / b

• Bash(pytest)
  L ===== test session starts =====
      platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
      rootdir: C:\Users\ashle\ada-02-agent-loop
      collected 4 items
```

```
test_calculator.py .... [100%]

===== 4 passed in 0.02s ===== (ctrl+o to collapse)

### Summary of Actions

1. Inspected Files: Reviewed calculator.py and test\_calculator.py.
2. Initial Test Run: Executed pytest before making changes.
   • Result: 1 failed, 3 passed (failure in test\_calculator.py:12-13 where divide(10, 2) evaluated to 20 instead of 5).
3. Applied Minimal Fix: Updated the calculator.py:10-11 function in calculator.py:10-11.
4. Validation: Re-ran pytest.
   • Result: 4 passed in 0.02s.

---

### What Was Changed

In calculator.py:10-11, corrected the operator in calculator.py:10-11 from multiplication (*) to division (/):

def divide(a, b):
-     return a * b
+     return a / b
```

En las imágenes se puede observar el ciclo que siguió el agente:

- **Inspección:** El agente primero inspeccionó los archivos relevantes del repositorio para identificar dónde podía encontrarse el bug. Utilizó la herramienta Read para leer el contenido de `calculator.py` y `test_calculator.py`. Al revisar los archivos, pudo identificar la implementación de la función `divide()` y la prueba correspondiente, que esperaba que `divide(10, 2)` devolviera 5.
- **Tool call:** Antes de realizar cualquier modificación, ejecutó `pytest`, tal como indicaban los requisitos.
- **Resultado:** El resultado fue 1 prueba fallida y 3 exitosas. La prueba de `divide()` falló porque la función devolvía 20 en lugar de 5. Esto permitió identificar que la función estaba utilizando el operador `*` en lugar de `/`.
- **Nueva acción:** El agente realizó el cambio mínimo necesario en `calculator.py`, sustituyendo `return a * b` por `return a / b`. No modificó los archivos de prueba.
- **Verificación:** Después de realizar el cambio, volvió a ejecutar `pytest` para comprobar que la corrección funcionara. Esta vez las 4 pruebas pasaron correctamente.

6. Experimento 3 — Nueva funcionalidad.

```
> Add a modulo operation to the calculator.
First inspect the repository.
Create or update the necessary tests.
Implement the feature.
Run the tests.
Do not make unrelated changes.

• Read(~/.ada-02-agent-loop/calculator.py)
  L Read 11 lines
• Read(~/.ada-02-agent-loop/test_calculator.py)
  L Read 13 lines
• Edit(~/.ada-02-agent-loop/test_calculator.py)
  L +1 / -1 lines

2
3     def test_add():
4         assert add(2, 3) == 5
• Edit(~/.ada-02-agent-loop/test_calculator.py)
  L +4 / -1 lines
10     assert multiply(4, 3) == 12
11
12     def test_divide():
13 -         assert divide(10, 2) == 5

16 +         assert modulo(10, 3) == 1
• Edit(~/.ada-02-agent-loop/calculator.py)
  L +4 / -1 lines
8         return a * b
9
10     def divide(a, b):
11 -         return a / b
```

```

16 +     assert modulo(10, 3) == 1
• Edit(~/ada-02-agent-loop/calculator.py)
  L +4 / -1 lines
      8         return a * b
      9
     10     def divide(a, b):
     11         return a / b

     14 +     return a % b
• Bash(pytest)
  L ===== test session starts =====
      platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
      rootdir: C:\Users\ashle\ada-02-agent-loop
      collected 5 items

      test_calculator.py ..... [100%]

      ===== 5 passed in 0.02s ===== (ctrl+o to collapse)

  ### Summary of Changes

  1. Inspected the Repository: Examined calculator.py and test_calculator.py.
  2. Updated Tests: Added test_calculator.py:15-16 and updated the import list in test_calculator.py:1.
  3. Implemented Feature: Added the calculator.py:13-14 function to calculator.py:13-14.
  4. Ran Tests: Executed pytest and all 5 tests passed successfully.

  ### Code Diffs

  **calculator.py**

      def divide(a, b):
          return a / b
      +
      +def modulo(a, b):
      +    return a % b

  **test_calculator.py**

      -from calculator import add, subtract, multiply, divide
      +from calculator import add, subtract, multiply, divide, modulo

      def test_divide():
          assert divide(10, 2) == 5
      +
      +def test_modulo():
      +    assert modulo(10, 3) == 1

```

7. Mini-Agent Loop.

```

• PS C:\Users\ashle\ada-02-agent-loop> python agent_loop.py

=== MINI-AGENT LOOP ===

Tarea: Find and fix the bug in the calculator.

Decisión:
{'tool': 'READ', 'args': {'filepath': 'calculator.py'}}

Observación:
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a * b

def modulo(a, b):
    return a % b

Decisión:
{'tool': 'READ', 'args': {'filepath': 'test_calculator.py'}}

Observación:
from calculator import add, subtract, multiply, divide, modulo

def test_add():
    assert add(2, 3) == 5

def test_subtract():
    assert subtract(5, 3) == 2

def test_multiply():
    assert multiply(4, 3) == 12

```

```
def test_divide():
    assert divide(10, 2) == 5

def test_modulo():
    assert modulo(10, 3) == 1

Decisión:
{'tool': 'BASH', 'args': {'command': 'pytest -q'}}

Observación:
...F. [100%]
===== FAILURES =====
test_divide
def test_divide():
>     assert divide(10, 2) == 5
E       assert 20 == 5
E       + where 20 = divide(10, 2)

test_calculator.py:13: AssertionError
===== short test summary info =====
FAILED test_calculator.py::test_divide - assert 20 == 5
1 failed, 4 passed in 0.05s

Decisión:
{'tool': 'EDIT', 'args': {'filepath': 'calculator.py', 'old_text': 'def divide(a, b):\n    return a * b', 'new_text': 'def divide(a, b):\n    return a / b'}}

Observación:
Archivo editado: calculator.py

Decisión:
{'tool': 'BASH', 'args': {'command': 'pytest -q'}}

Observación:
..... [100%]
5 passed in 0.01s

=== RESULTADO FINAL ===
El bug fue corregido y los tests pasaron.
PS C:\Users\ashle\ada-02-agent-loop>
```

8. Historial Git con commits.

Commits on Aug 31, 2026

Archivo agent-run.md que documenta las iteraciones principales del experimento 265a2679

ShadenAguilarM committed 2 minutes ago

Actualizar el archivo compilado de la calculadorab64c319

ShadenAguilarM committed 19 minutes ago

Creación del README con las preguntas obligatorias35cf3ec

ShadenAguilarM committed 21 minutes ago

Creación del mini-agent loop4dba85b

ShadenAguilarM committed 23 minutes ago

Commits on Aug 30, 2026

Nueva operación de módulo para la calculadora1da4673

ShadenAguilarM committed 18 hours ago

Initial calculator project2ec574f

ShadenAguilarM committed yesterday